# Sistema de Inventário e Hotbar no Godot 4

*Antigravity (Senior Game Developer)*

Projeto Poverty - Guia Técnico e Referência de Métodos

## 1. Guia Técnico Completo de Arquitetura, Implementação e Referência de Métodos

**Autor:** Antigravity (Senior Game Developer)
**Projeto:** Poverty (Godot 4 - 3D)
**Plataforma de Leitura:** Otimizado para leitura em Smartphones e Desktop

## 2. 1. Visão Geral e Filosofia de Design

No desenvolvimento profissional de jogos com o Godot 4, a separação estrita entre **Modelo de Dados (Model)** e **Interface do Usuário (View/Controller)** é indispensável. Misturar variáveis de contagem e referências de itens diretamente dentro de botões ou contêineres visuais resulta em problemas graves de dessincronização, bugs ao empilhar itens e código frágil.

### 2.1. Os Pilares do Sistema:

- **Dados Imutáveis e Compartilhados via Resource (ItemData):** Os tipos de itens (ex: Rifle, Munição, Kit Médico, Chave) são recursos salvos no disco (**.tres**). Eles armazenam propriedades estáticas como nome, ícone, cor e regras de agrupamento.
- **Estado Efêmero em Memória via RefCounted (InventorySlotData):** Cada slot no inventário ou na hotbar guarda apenas uma instância leve de dados com a referência ao item e a quantidade atual.
- **Componentização Modular da UI (InventorySlotUI):** Cada célula visual é independente, cuidando do desenho de bordas, badges de quantidade, ícones e disparando eventos de clique para o pai.
- **Gerenciador Centralizado de Estado (InventoryUI):** Orquestra as transições visuais (abrir/fechar com tecla E), atalhos de mouse (rolagem da hotbar, clique esquerdo, clique direito e Shift+Clique), e o cursor flutuante.
- **Integração Física e de Gameplay:** O jogador pode continuar andando com WASD enquanto o inventário está aberto, porém a mira da câmera e os disparos de tiro são congelados para permitir interação com o cursor do mouse. A arma 3D equipada na mão reage instantaneamente ao slot selecionado na Hotbar.

## 3. 2. Requisitos Atendidos em Detalhes

### 3.1. Grade de 6 Colunas por 3 Linhas (18 Slots)

O inventário principal é formatado em um **GridContainer** com 6 colunas, totalizando 18 espaços para armazenar itens.

### 3.2. Hotbar de 6 Slots e Foco com Rolagem do Mouse

A Hotbar possui exatamente 6 posições, alinhando-se com a largura das 6 colunas do inventário. A rotação do scroll do mouse (**WHEEL_UP** e **WHEEL_DOWN**) altera o slot focado de 0 a 5 com wrap-around contínuo (usando a função **posmod**). O slot ativo recebe uma moldura dourada destacada.

### 3.3. A Mágica da Transição da Hotbar

- **Inventário Fechado:** A Hotbar de HUD fica ancorada na parte inferior da tela.
- **Inventário Aberto (Tecla E):** A Hotbar inferior desaparece completamente e surge acoplada logo abaixo das 3 linhas e 6 colunas do inventário, aproximando os slots para facilitar a transferência e o arrasto de itens.

### 3.4. Controle de Agrupamento (‘can_stack‘ e ‘max_stack‘)

Cada item possui o booleano **can_stack**. Se for falso, o item não se agrupa (ex: Rifle e Chave). Se for verdadeiro, o limite padrão é acumular até 20 unidades sozinho (ex: Munição e Kit Médico).

### 3.5. Clique Direito (Extrair 1 Unidade)

Ao clicar com o botão direito sobre um slot com itens agrupados, exatamente 1 unidade é transferida para o cursor. Cliques com o botão direito adicionais sobre o mesmo item no slot somam +1 ao cursor até atingir o limite de 20. Se o cursor já tiver itens e o clique direito for em um slot vazio, 1 unidade é depositada nele.

### 3.6. Atalhos de Shift (‘Shift + Clique‘)

- **Clicar na Hotbar segurando Shift:** O item é movido automaticamente para o inventário.
- **Clicar no Inventário segurando Shift:** O item é movido automaticamente para a hotbar.
- **Lógica de Preenchimento Inteligente:** O algoritmo tenta primeiro completar pilhas incompletas existentes do mesmo item no destino (até 20) e depois procura o primeiro slot totalmente livre em ordem. Se não houver espaço, o comando não faz nada.

### 3.7. Suporte Híbrido: Minecraft (Clique-a-Clique) + Drag & Drop

O jogador pode tanto clicar no slot de origem, mover o mouse e clicar no slot de destino, quanto simplesmente segurar o botão esquerdo e arrastar até o outro slot. Ambos os fluxos funcionam nativamente.

## 4. 3. Explicação dos Scripts Criados e Modificados

### 4.1. 3.1. ‘script/item_data.gd‘ (‘ItemData‘)

Estende **Resource**. Representa o "molde" de cada item.

- id: Identificador textual único do item.
- item_name: Nome exibido na interface.
- icon: Textura gráfica exibida nos slots e no cursor flutuante.

- icon_modulate: Multiplicador de cor para o ícone.
- can_stack: Booleano que define se pode ser agrupado.
- max_stack: Limite máximo de empilhamento (padrão 20).
- is_rifle: Identificador para acoplamento do rifle 3D na mão do jogador.

### 4.2. 3.2. ‘script/inventory_slot_data.gd‘ (‘InventorySlotData‘)

Estende **RefCounted**. Representa a informação armazenada em cada posição de inventário ou no cursor.

- item: Referência ao ItemData presente no slot.
- quantity: Quantidade numérica de unidades presentes.

### 4.3. 3.3. ‘script/inventory_slot_ui.gd‘ (‘InventorySlotUI‘)

Estende **PanelContainer**. Representa visualmente um quadradinho na tela.

- Renderiza 3 estilos visuais com StyleBoxFlat (Normal, Hover e Focused).
- Controla visibilidade de ícone, nome e número de quantidade (ocultando quando quantidade for 1 ou vazio).
- Detecta eventos do mouse e emite o sinal slot_pressed.

### 4.4. 3.4. ‘script/inventory_ui.gd‘ (‘InventoryUI‘)

Estende **Control**. O maestro da interface.

- Contém os arrays de dados inventory_slots (18 posições) e hotbar_slots (6 posições).
- Mantém o objeto held_slot que representa o item preso ao cursor.
- Processa inputs da tecla E (inventory), rolagem do mouse e soltura de cliques.
- Controla a visibilidade e o posicionamento do cursor flutuante.

### 4.5. 3.5. ‘script/player.gd‘ (Integração)

- Escuta os sinais equipped_item_changed e inventory_toggled.
- Quando o inventário abre, libera o ponteiro do mouse (MOUSE_MODE_VISIBLE), oculta a mira (ColorRect) e trava o giro da visão no _input, mantendo a física de caminhada no _physics_process.
- Exibe ou esconde o nó 3D do Rifle dependendo se o slot selecionado na Hotbar contém o item do Rifle.

### 4.6. 3.6. ‘script/camera.gd‘ (Segurança de Disparo)

- Verifica se o inventário está fechado, se o mouse está capturado e se a arma está visível antes de chamar o método shoot().

## 5. 4. ADENDO ESPECIAL: Dicionário Detalhado de CADA Método Criado

Abaixo encontra-se a lista completa e a explicação técnica detalhada de cada método criado no sistema.

### 5.1. 4.1. Classe ‘ItemData‘ (‘script/item_data.gd‘)

- get_effective_max_stack() -> int: Retorna o valor efetivo de empilhamento. Se **can_stack** for falso, retorna obrigatoriamente **1**. Se for verdadeiro, retorna o valor de **max_stack** (padrão 20). Essa guarda previne que itens não agrupáveis recebam mais de uma unidade por engano.

### 5.2. 4.2. Classe ‘InventorySlotData‘ (‘script/inventory_slot_data.gd‘)

- _init(p_item: ItemData = null, p_quantity: int = 0) -> void: Construtor da classe. Permite inicializar o slot já com um item e quantidade definidos ou vazio por padrão.
- is_empty() -> bool: Verifica se o slot está desocupado. Retorna **true** se **item == null** ou se **quantity <= 0**.
- clear() -> void: Esvazia o slot, setando **item = null** e **quantity = 0**.
- set_data(p_item: ItemData, p_quantity: int) -> void: Define com segurança os dados do slot. Caso o item passado seja nulo ou a quantidade seja menor ou igual a 0, chama automaticamente **clear()**.
- copy_from(other: InventorySlotData) -> void: Clona os dados de outro slot para este, copiando a referência do item e a quantidade. Se o outro slot for nulo ou vazio, limpa este slot.
- can_stack_with(other_item: ItemData) -> bool: Retorna **true** se este slot contiver o mesmo item passado, se esse item permitir agrupamento (**can_stack == true**) e se a quantidade atual ainda não tiver atingido o limite máximo de empilhamento (**quantity < max_stack**).

### 5.3. 4.3. Classe ‘InventorySlotUI‘ (‘script/inventory_slot_ui.gd‘)

- _ready() -> void: Método padrão do ciclo de vida do Godot. Configura os estilos visuais de caixa (**_setup_styles**), conecta os sinais de entrada e saída do mouse (**mouse_entered** e **mouse_exited**) e aplica o estilo inicial.
- _setup_styles() -> void: Cria programaticamente 3 instâncias de **StyleBoxFlat**:
- Estilo normal: cinza escuro translúcido com borda suave de 2px.
- Estilo hover: fundo e borda ligeiramente iluminados ao passar o mouse.
- Estilo focado: fundo dourado escuro com **borda amarela brilhante de 3px** para indicar o slot ativo da Hotbar.
- update_slot(data: InventorySlotData, focused: bool = false) -> void: Recebe os dados do slot correspondente e atualiza a apresentação gráfica. Se estiver vazio, oculta os nós filhos de texto e textura. Se possuir item, preenche a textura do ícone, cor de modulação, nome do item no topo e contador de quantidade no canto inferior (apenas se a quantidade for maior que 1).
- _update_style() -> void: Aplica no tema do **PanelContainer** o **StyleBoxFlat** correto dependendo dos estados booleanos **is_focused** e **is_hovered**.
- _on_mouse_entered() -> void: Callback do sinal disparado quando o cursor do mouse entra na área retangular deste slot. Ativa a flag **is_hovered = true** e redesenha o estilo.
- _on_mouse_exited() -> void: Callback do sinal disparado quando o cursor do mouse sai do retângulo do slot. Desativa a flag **is_hovered = false** e redesenha o estilo.
- _gui_input(event: InputEvent) -> void: Intercepta os eventos de interface gráfica direcionados ao slot. Se detectar um clique pressionado com botão esquerdo ou direito, emite o sinal personalizado **slot_pressed** com o índice do slot, se é hotbar ou inventário, qual botão foi clicado e se o Shift estava pressionado.

### 5.4. 4.4. Classe ‘InventoryUI‘ (‘script/inventory_ui.gd‘)

- _ready() -> void: Ponto de partida do controlador de UI. Inicializa os arrays de dados (**_init_data**), gera os nós de interface instanciando a cena do slot (**_build_ui_slots**), popula os itens de demonstração (**_populate_starter_items**) e define o estado fechado inicial (**_set_open_state(false)**).
- _process(_delta: float) -> void: Atualiza a posição do nó **CursorItem** a cada frame renderizado para que ele siga com perfeição as coordenadas globais do mouse (**get_global_mouse_position()**) quando houver um item preso ao cursor.
- _input(event: InputEvent) -> void: Escuta inputs globais:
- Abertura/fechamento com a tecla E (event.is_action_pressed("inventory")).
- Rolagem do scroll do mouse para cima e para baixo para trocar o slot focado na hotbar.
- Soltura do botão esquerdo do mouse para completar ações de arrastar e soltar (Drag & Drop).
- _init_data() -> void: Preenche os arrays **inventory_slots** com 18 instâncias limpas de **InventorySlotData** e **hotbar_slots** com 6 instâncias limpas.
- _build_ui_slots() -> void: Configura as colunas das grades (6 colunas) e instancia dinamicamente 18 nós **InventorySlotUI** para o inventário, 6 para a hotbar no modal e 6 para a hotbar de HUD, conectando o sinal **slot_pressed** de cada um ao método centralizador **_on_slot_pressed**.
- _populate_starter_items() -> void: Adiciona itens de demonstração para testes imediatos (Rifle e Munição na Hotbar, e kits médicos, munição extra e chave no inventário).
- toggle_inventory() -> void: Inverte o estado atual do inventário (se aberto, fecha; se fechado, abre), chamando **_set_open_state**.
- _set_open_state(open: bool) -> void: Aplica as transformações visuais e de contexto do jogo:
- Se for fechar e houver item no cursor, devolve o item para o inventário com _return_held_item_on_close().
- Alterna a visibilidade da Hotbar inferior (hud_hotbar_panel.visible = not open) e da janela do inventário (inventory_modal.visible = open).
- Altera o modo do mouse entre capturado e visível (Input.MOUSE_MODE_VISIBLE / Input.MOUSE_MODE_CAPTURED).
- Atualiza todos os slots e emite o sinal inventory_toggled.
- _change_hotbar_focus(step: int) -> void: Altera o índice **selected_hotbar_index** somando o passo (**+1** ou **-1**) e aplicando a função matemática **posmod** com **HOTBAR_SIZE** (6). Atualiza os slots e emite **equipped_item_changed**.
- get_selected_hotbar_item() -> ItemData: Função de consulta rápida. Retorna o **ItemData** presente no slot ativo da Hotbar ou **null** se o slot estiver vazio.
- _on_slot_pressed(slot_index: int, is_hotbar: bool, button_index: int, shift_pressed: bool) -> void: Direcionador principal de cliques:
- Se o inventário estiver fechado, ignora.
- Se shift_pressed == true, chama _handle_shift_click.
- Se botão esquerdo, chama _handle_left_click.
- Se botão direito, chama _handle_right_click.
- Ao final, chama _refresh_all_ui().
- _handle_left_click(target_slot: InventorySlotData, slot_index: int, is_hotbar: bool) -> void: Trata o clique com botão esquerdo:
- Se o cursor estiver vazio e o slot tiver item: transfere todo o conteúdo para o cursor e inicia a marcação de arrasto (_is_dragging_left = true).

- Se o cursor já tiver item: chama _place_or_swap_held_into_slot(target_slot).
- _handle_mouse_left_release() -> void: Executado quando o botão esquerdo é solto. Varre os retângulos globais dos slots visuais com **has_point(mouse_pos)**. Se o jogador tiver soltado sobre outro slot diferente do original, deposita ou troca o item naquele slot imediatamente, concretizando o Drag & Drop.
- _place_or_swap_held_into_slot(target_slot: InventorySlotData) -> void: Lógica matemática de transferência:
- Se o slot destino estiver vazio: coloca tudo e limpa o cursor.
- Se tiver o mesmo item e for agrupável: calcula o espaço até o limite de 20 e transfere o que couber. Se atingir 20, a sobra continua no cursor.
- Se os itens forem diferentes ou não agrupáveis: faz a troca mútua (swap).
- _handle_right_click(target_slot: InventorySlotData) -> void: Lógica do botão direito:
- Se o slot clicado tiver item e o cursor estiver vazio: tira exatamente 1 unidade para o cursor.
- Se o cursor já tiver o mesmo item agrupável e < 20: retira mais +1 unidade do slot e soma ao cursor.
- Se o slot clicado for vazio e o cursor tiver itens: deposita 1 unidade do cursor no slot.
- _handle_shift_click(source_slot: InventorySlotData, from_hotbar: bool) -> void: Lógica de transferência expressa:
- Define o array destino (se veio da hotbar, vai para inventory_slots; se veio do inventário, vai para hotbar_slots).
- Passo 1: Se agrupável, procura pilhas existentes com o mesmo item no destino e completa até 20.
- Passo 2: Se ainda sobrar unidades, procura o primeiro slot com is_empty() == true e coloca o item lá.
- _return_held_item_on_close() -> void: Garantia de segurança: se o jogador fechar o inventário segurando um item no cursor, devolve para o slot de origem ou usa a lógica de shift-click para guardá-lo.
- _refresh_all_ui() -> void: Sincroniza todos os 18 nós do inventário, os 6 da hotbar modal e os 6 da hotbar HUD com seus respectivos dados, atualiza a visibilidade e o texto do cursor flutuante e emite **equipped_item_changed**.

### 5.5. 4.5. Métodos Integrados em ‘Player‘ (‘script/player.gd‘)

- is_inventory_open() -> bool: Retorna se o inventário está atualmente aberto consultando **inventory_ui.is_open**.
- _on_equipped_item_changed(item: ItemData) -> void: Receptor do sinal da hotbar. Se **item != null and item.is_rifle**, torna o nó 3D do Rifle visível na mão; caso contrário, esconde o rifle.
- _on_inventory_toggled(open: bool) -> void: Receptor do sinal de alternância do inventário. Oculta a mira central (**crosshair.visible = not open**) quando o inventário está aberto.